Лабораторная работа №3

Метрики качества для задач классификации

Цель лабораторной работы - изучить основные метрики качества для бинарной и многоклассовой классификации и научиться сравнивать несколько моделей по набору метрик, а также по ROC- и PR-кривым.

Задачи:

- ознакомиться с определениями TP, FP, TN, FN;
- разобрать метрики accuracy, precision, recall, specificity (TNR), F1 и их смысл;
- разобрать ROC AUC и PR AUC, построить ROC- и PR-кривые;
- на практике обучить и сравнить несколько классификаторов на бинарной задаче и на многоклассовой задаче;

Теоретическая часть

Метрики качества для задач классификации: точность и специфичность, F1, ROC AUC, PR AUC

Рассмотрим такие метрики, как точность (presicion), специфичность (specificity, true negative rate, TNR), F1-мера (F1-score), ROC AUC и PR AUC для бинарной и многоклассовой классификаций. Также рассмотрим долю правильных ответов (accuracy), которую часто путают с точностью.

Обозначения:

- $\mathbf{y} = (y_1, \ldots, y_n)$ — правильные ответы
- В задаче бинарной классификации считаем, что $y_i \in \{0, 1\}$ для любого $i \in [1, n]$
- $\hat{\mathbf{y}} = (\hat{y_1}, \ldots, \hat{y_n})$ — предсказания меток классов
- $\mathbf{p_i} = (p_1, \ldots, p_K)$ — предсказания вероятностей принадлежности к классам $\{C_1, \ldots, C_K\}$ для любого $i \in [1, n]$
- В задаче бинарной классификации считаем, что

$$p_1 = p \text{ – вероятность принадлежности к классу } 1$$
$$p_0 = 1 - p \text{ – вероятность принадлежности к классу } 0$$

Точность, доля правильных ответов (accuracy)

Формула расчета **доли правильных ответов** выглядит одинаково и в случае бинарной, и в случае многоклассовой классификации (единственное, в многоклассовом случае ее можно считать как для всех классов в целом, так и для каждого класса по отдельности, а потом усреднить полученные значения):

$$\text{accuracy} = \frac{1}{n} \sum_{i=1}^{n} 1(\hat{y_i} == y_i),$$

где $1(x)$ — **индикаторная функция**.

Пусть $A \subseteq X$ — выбранное подмножество произвольного множества X. Функция $1_A : X \to \{0, 1\}$, определенная следующим образом:

$$1_A(x) = \begin{cases} 1, & x \in A \\ 0, & x \notin A, \end{cases}$$

называется **индикатором** множества A.

+ Простая в понимании и интерпретации метрика.
– Неприменима, когда выборка несбалансированная (если в выборке будет очень мало объектов, представляющих какой-то класс, то даже константная модель (предсказания которой являются константой) покажет достаточно большую долю правильных ответов.

Реализация в `sklearn`

Точность (presicion) и специфичность (specificity/

true negative rate, TNR)

Бинарная классификация

Перед тем как переходить к формулам непосредственно точности и специфичности, введем следующие обозначения, которые лучше всего записать в виде таблицы:

	$y = 0$	$y = 1$
$\hat{y} = 0$	True negative (TN)	False negative (FN)
$\hat{y} = 1$	False positive (FP)	True positive (TP)

В данной таблице приняты следующие обозначения для объектов, попавших в каждую из ячеек:

- TN — модель правильно классифицировала объект, и он принадлежит классу 0
- FN — модель неправильно классифицировала объект, присвоила ему класс 0, хотя он принадлежит классу 1
- FP — модель неправильно классифицировала объект, присвоила ему класс 1, хотя он принадлежит классу 0

- TP — модель правильно классифицировала объект, и он принадлежит классу 1

В терминах, введенных выше, **точность (precision)** и **специфичность (specificity/TNR)** выглядят следующим образом:

$$\text{precision} = \frac{TP}{TP + FP}$$

– доля объектов, названных классификатором положительными и при этом действительно являющиеся положительным

$$\text{TNR} = \frac{TN}{TN + FP}$$ – показывает, какую долю объектов класса 0 из всех объектов класса 0 нашел алгоритм.

Также весьма распространена метрика **полнота (recall)**, которая является «братом-близнецом» специфичности:

$$\text{recall} = \frac{TP}{TP + FN}$$ – показывает, какую долю объектов класса 1 из всех объектов класса 1 нашел алгоритм.

Визуализировать различие между точностью и полнотой можно следующим образом:

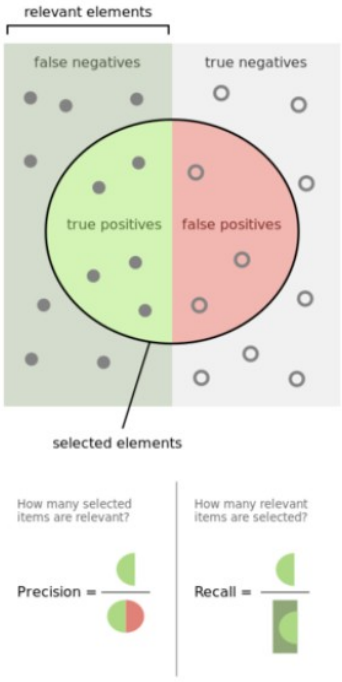

Рис. 1. Визуальное сравнение точности и полноты

Точность (precision) демонстрирует способность отличать класс 1 от других классов, а **полнота (recall)** — способность алгоритма обнаруживать класс 1 вообще (**специфичность** делает то же самое, что и полнота, только для класса 0).

Аналогичным образом, через TN, FN, FP и TP, можно ввести и понятие **доли правильных ответов**: $\text{accuracy} = \frac{TP+TN}{TP+FP+TN+FN}$.

+ Как точность, так и специфичность и полнота не зависят, в отличие от доли правильных ответов, от соотношения классов, поэтому они применимы и для несбалансированных выборок.
– На практике обычно не стоит задача оптимизировать какую-то одну из этих метрик, а необходимо найти баланс между ними.

Многоклассовая классификация

Как и в случае доли правильных ответов, существуют два типа агрегации этих метрик в многоклассовом случае:

- **Микро-** — необходимые значения вычисляются глобально, целиком по всей выборке, рассматривая каждый элемент матрицы показателей метки как метку
- **Макро-** — метрика считается отдельно для каждого класса, и в качестве ответа берется их среднее значение

Реализации (и для бинарного, и для многоклассового случая):

- Точность (precision)
- Полнота (recall)
- Специфичность (TRN). Реализации именно для специфичности нет, но можно получить из confusion matrix все необходимые значения (TN, TP, FN, и FP)

F_1-мера

Как упоминалось ранее, обычно важна не только точность или полнота, но и баланс между ними. Наиболее распространенной метрикой такого баланса является F_1**-мера** — среднее гармоническое точности и полноты:

$$F_1 = \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

F_1**-мера** достигает максимума при полноте и точности, равных единице, и близка к нулю, если один из аргументов близок к нулю.

Как в случае точности, специфичности и полноты, формула верна и для бинарной, и для многоклассовой классификации, однако в многоклассовом случае необходимо уточнить вид агрегации: микро- (когда показатели считаются глобально для всей выборки) или макро- (точность и полнота считаются отдельно для каждого класса, а в качестве ответа берется среднее). Реализация в `sklearn`

Существуют и другие варианты соотношений точности и полноты. В целом F_1**-мера** — частный случай F_β**–меры**:

$$F_\beta = (1 + \beta^2)\frac{\text{precision} \cdot \text{recall}}{\beta^2 \cdot \text{precision} + \text{recall}}$$

- $\beta > 1$ — больше важна точность
- $\beta < 1$ — больше важна полнота

+ F_1-мера учитывает распределение классов (т. е. хорошо работает даже в случае несбалансированных выборок).
+ Это одно число, а не два, как с точностью и полнотой.
– Сложно интерпретируемая.

ROC AUC

Бинарная классификация

При получении предсказаний алгоритмы выдают вероятности принадлежности к каждому классу.

Для принятия решения, к какому классу все-таки принадлежит конкретный элемент, необходимо выбрать такой порог T_p, что $p > T_p \implies$ объект принадлежит к классу 1, иначе — к классу 0. Интуитивно кажется, что наиболее оптимальным и логичным является порог $T_p = 0.5$, однако это не всегда так. Чтобы убрать привязку к какому-то конкретному пороговому значению, была введена метрика **ROC AUC** — площадь (Area Under Curve) под кривой ошибок (Receiver Operating Characteristic curve). Кривая ошибок строится в координатах **True Positive Rate (TPR)** и **False Positive Rate (FPR)**:

$$TPR = \frac{TP}{TP + FN}; \qquad FPR = \frac{FP}{FP + TN}.$$

Как видно из формулы, TPR — это полнота, а FPR — доля объектов класса 0, которым алгоритм присвоил метку 1. Кривые ошибок — красная, зеленая и черная кривые на графике ниже:

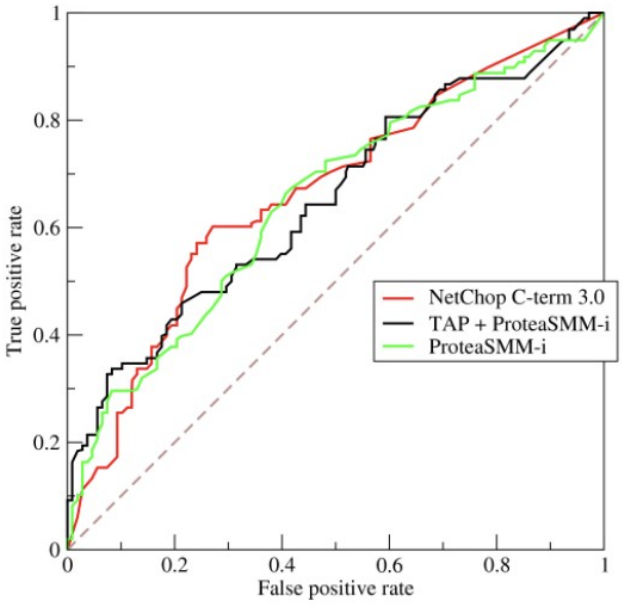

Рис. 2. ROC-кривая

В идеальном случае, когда классификатор предсказывает все верно, $FPR = 0, \quad TPR = 1 \implies ROC\ AUC = 1$. Если классификатор случайно выдает вероятности классов, $ROC\ AUC \to 0.5$ (поскольку классификатор будет выдавать одинаковое количество TP и FP).

+ Работает с ранжированными прогнозами, т. е. не обязательно использовать хорошо откалиброванные вероятности.
+ Оптимизация данной метрики помогает алгоритму учиться предсказывать два класса, т. е. исключается факт получения константной модели.
+ Для сравнения алгоритмов подходит не только площадь под ROC-кривой, но и сама кривая (чем ROC-кривая выше, тем лучше модель).
– Плохо работает с сильно несбалансированными выборками.

Реализация в `sklearn`

Многоклассовая классификация

В случае с многоклассовой классификацией наряду с макро- и микроагрегацией выделяются два следующих подхода в подсчете **ROC AUC**:

- **One vs Rest (OvR)**. Последовательно производится сравнение каждого класса (т. е. рассматривается как «положительный» или 1) с остальными (которые рассматриваются как «отрицательные» или 0). Для получения финального результата полученные таким образом метрики усредняются (либо с помощью простого усреднения, либо с помощью взвешивания). Иллюстрация на примере трех классов приведена ниже. image.png
  Рис. 3. Многоклассовая классификация — визуализация вычисления ROC AUC с помощью OvR
- **One vs One (OvO)**. Выглядит аналогичным образом, но сравниваются не один конкретный класс с остальными, как в случае OvR, а все возможные пары классов, т. е. каждый конкретный класс сравнивается с каждым другим классом. Иллюстрация на примере трех классов приведена ниже. image.png
  Рис. 4. Многоклассовая классификация — визуализация вычисления ROC AUC с помощью OvO

PR AUC

PR AUC — площадь (Area Under Curve) под кривой точности и полноты (Precision-Recall curve). Выглядит следующим образом:

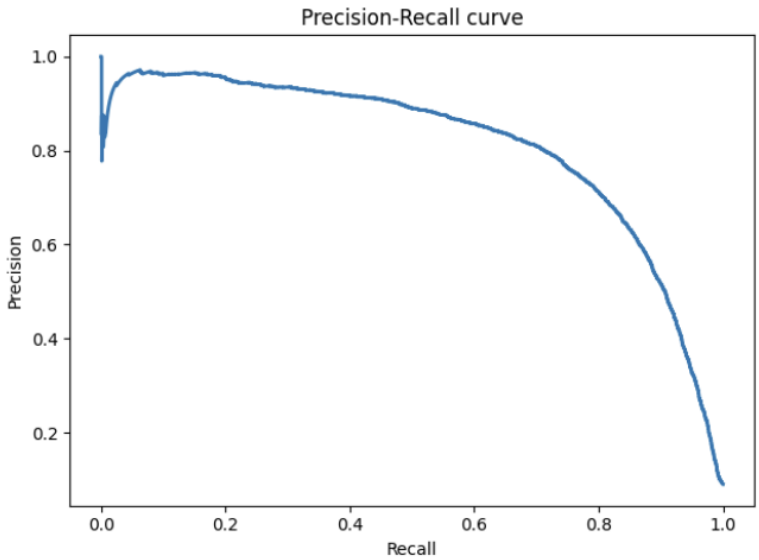

Рис. 5. PR-кривая

PR AUC помогает избавиться от проблемы с несбалансированной выборкой, возникающей при использовании ROC AUC. Рассмотрим пример: пусть имеются статьи на математические и гуманитарные темы. Допустим, что математических статей 100, а гуманитарных — 1000000. В любом случае идеальный алгоритм будет решать задачу с $TPR = 1$ и $FPR = 0$. Рассмотрим теперь «плохой» (что очевидно нам) алгоритм, дающий положительный ответ на 95 математических и 50000 нематематических статьях. Такой алгоритм дает $TPR = 0.95$ и $FPR = 0.05$, что крайне близко к показателям идеального алгоритма. В данном случае ROC AUC не может давать адекватную оценку качества работы алгоритма. Однако мы можем сказать, что

$$TN = 950000, \;\; FP = 50000, \; FN = 5, \; TP = 95 \;\; \Longrightarrow$$
$$\Longrightarrow \;\; \text{precision} = \frac{95}{95 + 50000} = 0.002 \quad \text{recall} = \frac{95}{95 + 950000} = 0.0001.$$

И при таких значениях кажется очевидным, что и при построении PR-кривой площадь под ней окажется очень маленькой, что будет свидетельствовать о том, что алгоритм плохой.

+ Помогает выбрать наиболее подходящий порог классификатора.
+ Хорошо работает и в случае несбалансированной выборки.

Как и в точности и полноте, в случае многоклассовой классификации используются аналогичные виды агрегации.

Подробнее о взаимосвязи с ROC AUC

Реализация в `sklearn`

```
round(95 / (95 + 50000), 3), round(95 / (95 + 950000), 5)
```

```
(0.002, 0.0001)
```

Практическая часть

Бинарная классификация

Для задачи бинарной классификации будем использовать данные с kaggle. Так же как и в семинаре про регрессионные модели, данные можно скачать двумя способами:

1. Скачать данные с Kaggle, в Google Colab нажать слева значок папки и перетащить данные туда
2. С помощью kaggle API

Скачивание данных с kaggle API

Ниже будут представлены ячейки, скачивающие данные с помощью kaggle API. Подробная инструкция, что делает каждая конкретная ячейка, приведена в семинаре по регрессии (модуль 2).

```
from google.colab import drive
drive.mount('/content/gdrive')
```

```
Mounted at /content/gdrive
```

```
from google.colab import files
files.upload()
```

Выбрать файлы Файл не выбран Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
{}
```

```
!ls -lha kaggle.json
```

```
ls: cannot access 'kaggle.json': No such file or directory
```

```
!pip install -q kaggle
```

```
!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
```

```
cp: cannot stat 'kaggle.json': No such file or directory
```

```
!chmod 600 /root/.kaggle/kaggle.json
```

```
chmod: cannot access '/root/.kaggle/kaggle.json': No such file or directory
```

```
!kaggle datasets download -d joshmcadams/oranges-vs-grapefruit
```

```
Traceback (most recent call last):
  File "/usr/local/bin/kaggle", line 5, in <module>
    from kaggle.cli import main
  File "/usr/local/lib/python3.8/dist-packages/kaggle/__init__.py", line 23, in <module>
    api.authenticate()
  File "/usr/local/lib/python3.8/dist-packages/kaggle/api/kaggle_api_extended.py", line 164, in authenticate
    raise IOError('Could not find {}. Make sure it\'s located in'
OSError: Could not find kaggle.json. Make sure it's located in /root/.kaggle. Or use the environment method.
```

Вне зависимости от способа скачивания и загрузки данных в Google Colab разархивируем их:

```
!unzip oranges-vs-grapefruit.zip
```

```
unzip:  cannot find or open oranges-vs-grapefruit.zip, oranges-vs-grapefruit.zip.zip or oranges-vs-grapefruit.zip
```

```
# Уберем предупреждения, чтобы они не загромождали вывод

import warnings
warnings.filterwarnings('ignore')
```

Разведочный анализ и предобработка данных

```
import pandas as pd

df = pd.read_csv("citrus.csv")
print(df.shape)
df.head()
```

```
---------------------------------------------------------------------------
FileNotFoundError                         Traceback (most recent call last)
<ipython-input-11-fb3896fab9d6> in <module>
      1 import pandas as pd
      2
----> 3 df = pd.read_csv("citrus.csv")
      4 print(df.shape)
      5 df.head()

7 frames
/usr/local/lib/python3.8/dist-packages/pandas/io/common.py in get_handle(path_or_buf, mode, encoding,
compression, memory_map, is_text, errors, storage_options)
    700         if ioargs.encoding and "b" not in ioargs.mode:
    701             # Encoding
--> 702             handle = open(
    703                 handle,
    704                 ioargs.mode,

FileNotFoundError: [Errno 2] No such file or directory: 'citrus.csv'
```

```
df.isna().sum()
```

Пропусков в данных нет, все признаки численные, поэтому из предобработки данных будет только нормализация признаков и кодирование целевой переменной. Посмотрим на целевую переменную.

```
df["name"].value_counts()
```

В данном случае классы идеально сбалансированы, поэтому все вышеперечисленные метрики должны быть показательны для наших алгоритмов.

```
# Поскольку кодируется целевая переменная, можно проводить кодировку
# сразу на всех данных (до деления выборки на обучающую и тестовую)

from sklearn.preprocessing import LabelEncoder

enc = LabelEncoder()
df["name"] = enc.fit_transform(df["name"])
df.head()
```

```
# Разбиение выборки на обучающую и тестовую

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    df.drop("name", axis=1),
    df["name"],
    test_size=0.33,
    random_state=42
)
```

```
# Нормализация

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

X_train.shape, X_test.shape, X_train_scaled.shape, X_test_scaled.shape
```

Будем сравнивать алгоритмы с гиперпараметрами по умолчанию, т. е. подбор гиперпараметров проводить не будем. По возможности будем использовать готовые реализации метрик из библиотеки `sklearn`.

Подготовка необходимых функций

```
from sklearn.metrics import (
    accuracy_score, # доля правильных ответов
    precision_score, # точность
    recall_score, # полнота
    confusion_matrix, # TN, TP, FN, FP -- для подсчёта TNR, точности и полноты для бинарной классификации
    multilabel_confusion_matrix, # возвращает список confusion_matrix (для многоклассового случая -- будет испол
    f1_score, # F1-мера
    roc_auc_score, # ROC AUC
    roc_curve, # возвращает FPR, TPR и пороговые значения, которые использовались при подсчёте FPR и TPR
    average_precision_score, # PR AUC
    precision_recall_curve # возвращает точность, полноту и пороговые значения, которые использовались при подсч
)
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
```

Функция `calculate_precision_recall_f1` необходима для вычисления точности, полноты и F1-меры по формулам, используя TN, FP, FN, и TP, приведенные в теоретической части. Данная функция будет использоваться в задаче бинарной классификации для сравнения этих метрик, полученных через библиотеку `sklearn`, с метриками, полученными в ходе вычислений по формулам (для проверки корректности).

```
def calculate_precision_recall_f1(tn, fp, fn, tp):
    """Подсчёт точности, полноты и F1-меры по формулам.

    :param tn: true negative
    :param fp: false positive
    :param fn: false negative
    :param tp: true positive
    :return: точность, полнота, F1-мера
    """
    precision = tp / (tp + fp)
    recall = tp / (tp + fn)
    f1 = 2 / (precision + recall) * precision * recall

    return precision, recall, f1
```

Поскольку готовой реализации вычисления специфичности нет, будем использовать функцию, написанную ниже. Для бинарной классификации будем вычислять специфичность согласно формуле из теоретической части, для многоклассовой — вариант макроагрегации (вычисление метрики для каждого класса отдельно с последующим усреднением всех полученных таким образом результатов).

```
import numpy as np

def calculate_specificity(true_labels, predictions):
    """Вычисление специфичности для бинарной и многоклассовой классификации.

    :param true_labels: истинные метки классов
    :param predictions: предсказанные метки классов
    :return: специфичность и confusion_matrix
    """
    if len(np.unique(true_labels)) < 3:
        conf_matrix = confusion_matrix(true_labels, predictions)
        tn, fp, fn, tp = conf_matrix.ravel()
        return tn / (tn + fp), (tn, fp, fn, tp)
    else:
        # в многоклассовом случае будем использовать только макро-аггрегацию
        conf_matrix = multilabel_confusion_matrix(true_labels, predictions)
        specificity = []

        for cm in conf_matrix:
            tn, fp, fn, tp = cm.ravel()
            specificity.append(tn / (tn + fp))
        return np.array(specificity).mean(), conf_matrix
```

Поскольку мы хотим вычислять много метрик для нескольких алгоритмов, их вычисление лучше всего поместить в отдельную функцию (например, такую, как приведена ниже).

```
def get_all_metrics(
    true_labels,
    predictions,
    probabilities,
    print_metrics=False,
    **kwargs,
):
    """Подсчёт и вывод всех метрик.

    :param true_labels: истинные метки классов
    :param predictions: предсказания (метки классов)
    :param probabilities: предсказания (вероятности принадлежности каждого объекта к каждому классу)
    :param print_metrics: если True, то печатает получившиеся метрики
    :return: словарь, ключи которого -- названия метрик, значения -- получившиеся метрики
    """
    kwargs_precision_recall_f1_pr_auc = dict() # для бинарной классификации никакие дополнительные параметры не
    kwargs_roc_auc = dict()

    # если бинарная классификация, то используем только вероятности принадлежности к классу 1
    if len(np.unique(true_labels)) < 3:
        probabilities = probabilities[:, 1]
        true_predicted_labels_precision_recall_f1 = {
            "y_true": true_labels,
            "y_pred": predictions,
        }
        true_predicted_labels_pr_auc = {
            "y_true": true_labels,
            "y_score": probabilities,
        }
    # если многоклассовая классификация, то устанавливаем дополнительные параметры для посчета метрик
    else:
        kwargs_precision_recall_f1_pr_auc = {
            "average": kwargs["average"],
        }
        kwargs_roc_auc = {
            "multi_class": kwargs["multi_class"]
        }
        true_labels_binarize = label_binarize(true_labels, classes=np.unique(true_labels))
        predictions_binarize = label_binarize(predictions, classes=np.unique(predictions))
        true_predicted_labels_precision_recall_f1 = {
            "y_true": true_labels_binarize,
            "y_pred": predictions_binarize,
        }
        true_predicted_labels_pr_auc = {
            "y_true": true_labels_binarize,
            "y_score": probabilities,
        }
    accuracy = accuracy_score(true_labels, predictions)

    precision = precision_score(**true_predicted_labels_precision_recall_f1, **kwargs_precision_recall_f1_pr_auc
    recall = recall_score(**true_predicted_labels_precision_recall_f1, **kwargs_precision_recall_f1_pr_auc)
    f1 = f1_score(**true_predicted_labels_precision_recall_f1, **kwargs_precision_recall_f1_pr_auc)

    roc_auc = roc_auc_score(true_labels, probabilities, **kwargs_roc_auc)
    pr_auc = average_precision_score(**true_predicted_labels_pr_auc, **kwargs_precision_recall_f1_pr_auc)

    specificity, conf_matrix = calculate_specificity(true_labels, predictions)

    if len(np.unique(true_labels)) < 3:
        # Для бинарной классификации сравним точность, полноту и F1-меру, полученные с помощью sklearn
        # с вычисленными по формулам данными метриками (просто дополнительно выведем их)
        tn, fp, fn, tp = conf_matrix
        precision_cm, recall_cm, f1_cm = calculate_precision_recall_f1(tn, fp, fn, tp)

        if print_metrics:
            print(
                "Accuracy = {}\nPrecision (from sklearn) = {}; Precision (from scratch) = {}\nRecall (from sklea
                    accuracy, precision, precision_cm, recall, recall_cm, f1, f1_cm, roc_auc, pr_auc, specificit
                )
            )

    else:
        if print_metrics:
            print(
                "Accuracy = {}\nPrecision = {}\nRecall = {}\nF1_score = {}\nROC AUC = {}\nPR AUC = {}\nSpecifici
                    accuracy, precision, recall, f1, roc_auc, pr_auc, specificity
                )
            )
```

```
    return {
        "Accuracy": accuracy,
        "Precision": precision,
        "Recall": recall,
        "F1_score": f1,
        "ROC AUC": roc_auc,
        "PR AUC": pr_auc,
        "Specificity": specificity
    }
```

Поскольку необходимо сравнивать несколько моделей, напишем отдельную функцию для обучения и тестирования модели. Также эта функция будет вычислять и все указанные выше метрики.

```
from sklearn.preprocessing import label_binarize

def model_train_test(X_train, y_train, X_test, y_test, model, **kwargs):
    """Обучение и тестирование модели на обучающей и тестовой выборке соответственно.

    :param X_train: объекты обучающей выборки
    :param y_train: значения целевой переменной обучающей выборки
    :param X_test: объекты тестовой выборки
    :param y_test: значения целевой переменной тестовой выборки
    :param model: модель, которая обучается и тестируется
    :return: предсказания -- классы и вероятности принадлежности к каждому классу, все метрики
    """
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    proba = model.predict_proba(X_test)

    print(str(model), end="\n\n")

    all_metrics = get_all_metrics(
        y_test,
        pred,
        proba,
        print_metrics=True,
        **kwargs,
    )
    return pred, proba, all_metrics
```

Сравнение алгоритмов

Перейдем к непосредственному сравнению моделей. В задаче бинарной классификации будем сравнивать логистическую регрессию, решающее дерево, метод k-ближайших соседей и метод опорных векторов.

Подробнее про метод опорных векторов здесь и здесь.

```
pred_logreg, proba_logreg, all_metrics_logreg = model_train_test(
    X_train_scaled, y_train.values,
    X_test_scaled, y_test.values,
    LogisticRegression()
)
```

```
pred_dt, proba_dt, all_metrics_dt = model_train_test(
    X_train, y_train.values,
    X_test, y_test.values,
    DecisionTreeClassifier()
)
```

```
pred_knn, proba_knn, all_metrics_knn = model_train_test(
    X_train_scaled, y_train.values,
    X_test_scaled, y_test.values,
    KNeighborsClassifier()
)
```

```
pred_svm, proba_svm, all_metrics_svm = model_train_test(
    X_train_scaled, y_train.values,
    X_test_scaled, y_test.values,
    SVC(probability=True)
)
```

Сравним не только ROC AUC и PR AUC, но и ROC- и PR-кривые. Для их визуализации будем использовать функцию, приведенную ниже:

```
import matplotlib.pyplot as plt

def plot_roc_pr_curves(true_labels, algorithms):
    """Отрисовка ROC- и PR-кривых.

    :param true_labels: истинные метки классов
    :param algorithms: словарь формата {название_алгоритма: вероятности_принадлежности_к_каждому_классу}
    """
    fig, ax = plt.subplots(1, 2, figsize=(20, 8))
    ax[0].set_title("ROC-curve", fontsize=20)
    ax[1].set_title("PR-curve", fontsize=20)

    for alg_name, probabilities in algorithms.items():
        fpr, tpr, _ = roc_curve(true_labels, probabilities[:, 1])
        precisions, recalls, _ = precision_recall_curve(true_labels, probabilities[:, 1])

        ax[0].plot(
            fpr,
            tpr,
            label="{} (ROC AUC = {})".format(
                alg_name,
                round(
                    roc_auc_score(true_labels, probabilities[:, 1]),
                    3
                )
            )
        )
        ax[1].plot(
            recalls,
            precisions,
            label="{} (PR AUC = {})".format(
                alg_name,
                round(
                    average_precision_score(true_labels, probabilities[:, 1]),
                    3
                )
            )
        )

    ax[0].legend(fontsize=14)
    ax[0].set_xlabel("False positive rate", fontsize=16)
    ax[0].set_ylabel("True positive rate", fontsize=16)

    ax[1].legend(fontsize=14)
    ax[1].set_xlabel("Recall", fontsize=16)
    ax[1].set_ylabel("Precision", fontsize=16)
    plt.show()
```

```
plot_roc_pr_curves(
    y_test,
    {
        "Logistic Regression": proba_logreg,
        "Decision Tree": proba_dt,
        "kNN": proba_knn,
        "SVM": proba_svm,
    }
)
```

Как упоминалось в теоретической части, можно сравнивать не только площади под ROC- и PR-кривыми, но и сами кривые. Чем выше кривая, тем лучше модель (верно и для ROC-кривой, и для PR-кривой).

Сагрегируем все метрики моделей.

```
ans_df = pd.DataFrame(
    [all_metrics_logreg, all_metrics_dt, all_metrics_knn, all_metrics_svm],
    index=["Logistic Regression", "Decision Tree", "kNN", "SVM"]
)
ans_df
```

Для каждой метрики выведем список моделей по убыванию их качества.

```
for metric in ans_df.columns:
    sorted_ans_df = ans_df.sort_values(by=metric, ascending=False)
    print(
        "{}:{}{}".format(
            metric,
            " " * (20 - len(metric)),
            sorted_ans_df.index.tolist()
```

```
        )
    )
```

Согласно всем метрикам наилучшим алгоритмом в данном случае оказалась логистическая регрессия. Хуже всего отработало решающее дерево (судя по ROC AUC и PR AUC) и метод k-ближайших соседей (судя по доли правильных ответов, точности, полноте, F1-мере и специфичности). Однако, посмотрев на численные значения всех метрик всех моделей, можно сказать, что они все обучились достаточно хорошо.

Многоклассовая классификация

Чтение и предобработка данных

```
from sklearn.datasets import load_wine
```

```
wine_data = load_wine()
wine_data.keys()
```

```
df = pd.DataFrame(wine_data["data"], columns=wine_data["feature_names"])
target = wine_data["target"]

df.head()
```

```
pd.Series(target).value_counts()
```

Выборка не идеально сбалансирована, но сильных различий в количестве представителей классов нет.

В данных представлены только численные признаки, поэтому единственный вариант предобработки данных, который можно сделать на этих данных, — нормализация. Однако мы будем использовать методы, в основе которых лежат деревья, поэтому даже нормализацию проводить не будем.

```
# Делим выборку на обучающую и тестовую

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(df, target, test_size=0.33, random_state=42)
```

Обучение и тестирование моделей

Поскольку существует несколько вариантов агрегации метрик в многоклассовом случае, перепишем функцию `model_train_test`. Новая функция `all_average_multi_class_combinations` обучает и тестирует модель, а также проводит подсчет метрик для различных вариантов агрегации метрик.

```
def all_average_multi_class_combinations(X_train, y_train, X_test, y_test, model, verbose=0):
    """После обучения и тестирования модели используются всевозможные аггрегации метрик.

    :param X_train: обучающая выборка
    :param y_train: целевая переменная обучающей выборки
    :param X_test: тестовая выборка
    :param y_test: целевая переменная тестовой выборки
    :param model: классификатор
    :param verbose: если 0, то выводятся только точность, полнота, F1-мера, ROC AUC и PR AUC для различных вариа
    :return: точность, полнота, F1-мера, ROC AUC и PR AUC для различных вариантов аггрегаций
    """
    # average="macro" и multi_class="ovr" -> ROC AUC чувствителен к несбалансированности классов
    # average="macro" и multi_class="ovo" -> ROC AUC не чувствителен к несбалансированности классов

    model.fit(X_train, y_train) # обучение модели
    pred = model.predict(X_test) # получение предсказаний (классов)
    proba = model.predict_proba(X_test) # получение предсказаний (вероятности принадлежности к каждому классу)

    if verbose > 0:
        print(str(model), end="\n\n")

    ans = dict()
    ans["ROC AUC"] = dict()
    metrics_to_compare = ["Precision", "Recall", "F1_score", "PR AUC"]
    for m in metrics_to_compare:
        ans[m] = dict()
```

```
    for average in ["macro", "micro", "weighted"]:
        for multi_class in ["ovr", "ovo"]:
            if verbose > 0:
                print("\n{}_{}".format(average, multi_class))

            # вычисление всех метрик
            all_metrics = get_all_metrics(
                y_test,
                pred,
                proba,
                calc_specificity=False,
                print_metrics=verbose > 0,
                average=average,
                multi_class=multi_class,
            )

            for m in metrics_to_compare:
                ans[m][average] = all_metrics[m]
            ans["ROC AUC"][multi_class] = all_metrics["ROC AUC"]
    if verbose > 0:
        print("-"*100, end="\n\n")
    for k, v in ans.items():
        print(k, "\n", v)
    return ans
```

Сравненмие моделей

В данной задаче будем сравнивать решающее дерево с одним из вариантов ансамблей решающих деревьев — случайным лесом.

```
dt = all_average_multi_class_combinations(
    X_train, y_train,
    X_test, y_test,
    DecisionTreeClassifier(),
    verbose=1,
)
```

Как видно из получившихся результатов, агрегация в данном случае не сильно повлияла на итоговое значение каждой из метрик.

```
from sklearn.ensemble import RandomForestClassifier
```

```
rf = all_average_multi_class_combinations(
    X_train, y_train,
    X_test, y_test,
    RandomForestClassifier(),
    verbose=1,
)
```

Случайный лес выдает почти идеальные предсказания, поэтому разница в агрегации результатов небольшая. Можем визуализировать каждое из деревьев, чтобы посмотреть, как достигаются почти идеальные предсказания.

```
from sklearn.tree import plot_tree

rf_clf = RandomForestClassifier()
rf_clf.fit(X_train, y_train)

fig = plt.figure(figsize=(15, 10))
plot_tree(
    rf_clf.estimators_[0], # визуализация дерева с индексом 0; всего len(rf_clf.estimators_) деревьев
    feature_names=wine_data.feature_names,
    class_names=wine_data.target_names,
    filled=True,
    impurity=True,
    rounded=True
);
```

Таким образом, для каждой задачи, исходя из ее особенностей, нужно выбирать модель и метрику качества. При выборе метрики качества лучше исходить из того, сбалансированная ли выборка.

Сбалансированная выборка:

- *Точность/доля правильных ответов (accuracy)*. При этом легко интерпретируемая метрика

- *ROC AUC*. Использует вероятности принадлежности к классам, которые не обязательно должны быть хорошо откалиброваны. Теоретически может работать и с неидеально сбалансированной выборкой, но при сильном дисбалансе классов становится нерепрезентативной

Несбалансированная выборка:

- *Точность (precision), полнота (recall), специфичность (specificity, TNR)*. Стоит учитывать, что обычно нужны не каждая из них по отдельности, а их баланс
- *F1-мера*. Является одним числом, среднее гармоническое точности и полноты
- *PR AUC*. Также является одним числом, но, кроме этого, может помочь выбрать порог классификатора

```
Напишите программный код или сгенерируйте его с помощью искусственного интеллекта.
```

Итоговые выводы (что отразить в отчете)

- какие метрики лучше подходят для сбалансированной и для несбалансированной выборки;
- какая модель показала лучшие ROC AUC и PR AUC на бинарной задаче;
- как меняются precision/recall/F1 при выборе micro/macro/weighted для многокласса;
- в чем смысл схем OvR и OvO при расчете ROC AUC.

Контрольные вопросы

1. Что такое классификация? Чем бинарная отличается от многоклассовой?
2. Что такое confusion matrix? Что означают TP, FP, TN, FN?
3. В чем разница между accuracy и precision?
4. Когда важнее recall, а когда precision? Приведите примеры.
5. Что такое specificity (TNR) и как ее интерпретировать?
6. Почему F1 удобнее, чем пара (precision, recall) по отдельности?
7. Что такое ROC-кривая и ROC AUC? Что означает ROC AUC = 0.5?
8. Почему PR AUC часто предпочтительнее ROC AUC при сильном дисбалансе классов?
9. Что такое micro/macro/weighted усреднение и когда что выбирать?
10. Чем отличаются OvR и OvO в многоклассовом ROC AUC?